Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №9

навчальної дисципліни

ПРОГРАМНІ МЕТОДИ ЗАХИСТУ ІНФОРМАЦІЇ

ФКЗЕ. 121ПМЗІ05. 09ЛР

Шифр роботи

Студент Захарчук Ю.О.

Група ПЗ-23-1/9

Перевірив Костенко А.О.

2026 - 2027

Тема: Програмні засоби захисту від атак з мережі Internet.

Мета: вивчити основні типи мережевих атак та програмні засоби захисту від них. Навчитися аналізувати мережевий трафік, налаштовувати брандмауер та виявляти вразливості.

Обладнання: ПК з встановленим Wireshark або tcpdump; Доступ до командного рядка з правами адміністратора; Компілятор або інтерпретатор мови програмування за вибором студента; Віртуальна машина (опціонально, для безпечних експериментів).

Варіант: 5.

Хід роботи

1 Отримати в викладача номер індивідуального варіанта.

2 Аналіз мережевого трафіку:

- Встановити Wireshark (якщо не встановлено)
- Запустити захоплення трафіку на основному інтерфейсі
- Відвідати кілька веб-сайтів (HTTP і HTTPS)
- Знайти та проаналізувати HTTP-запити
- Застосувати фільтри для виділення трафіку

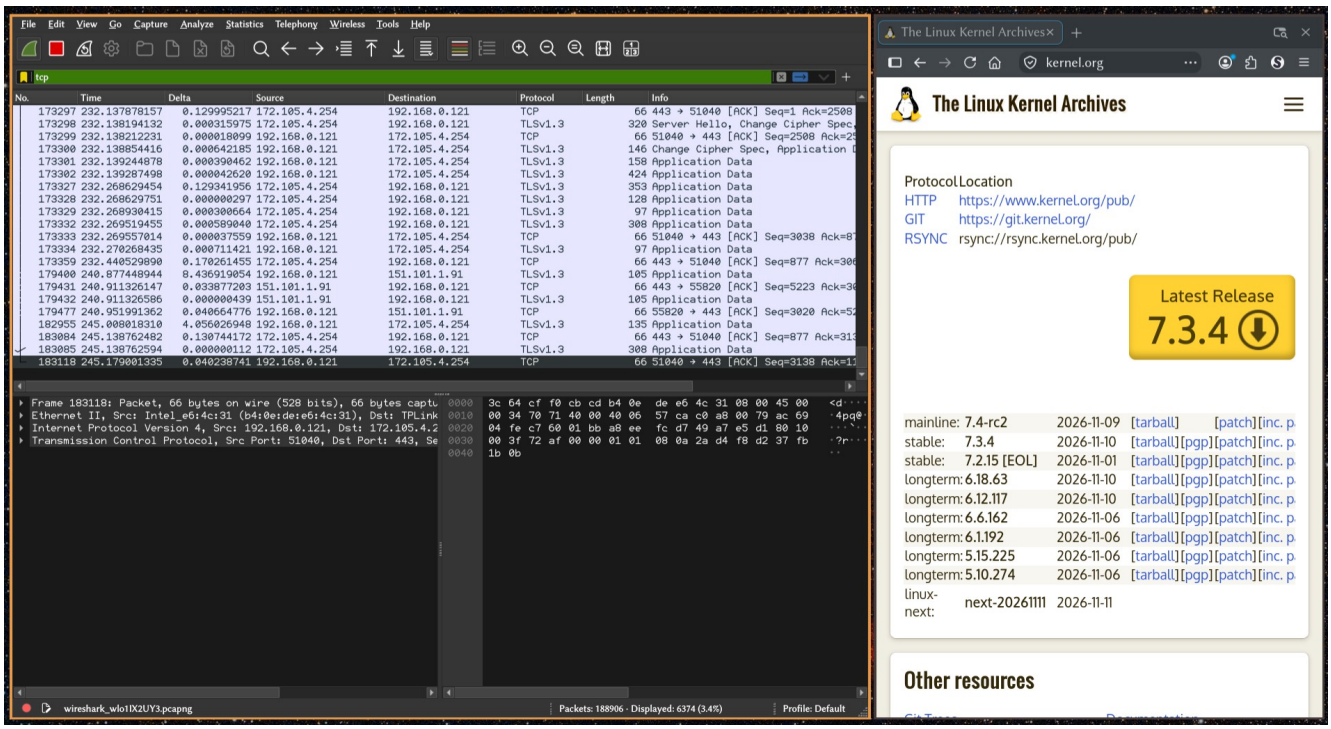


Рисунок 1 - https трафік

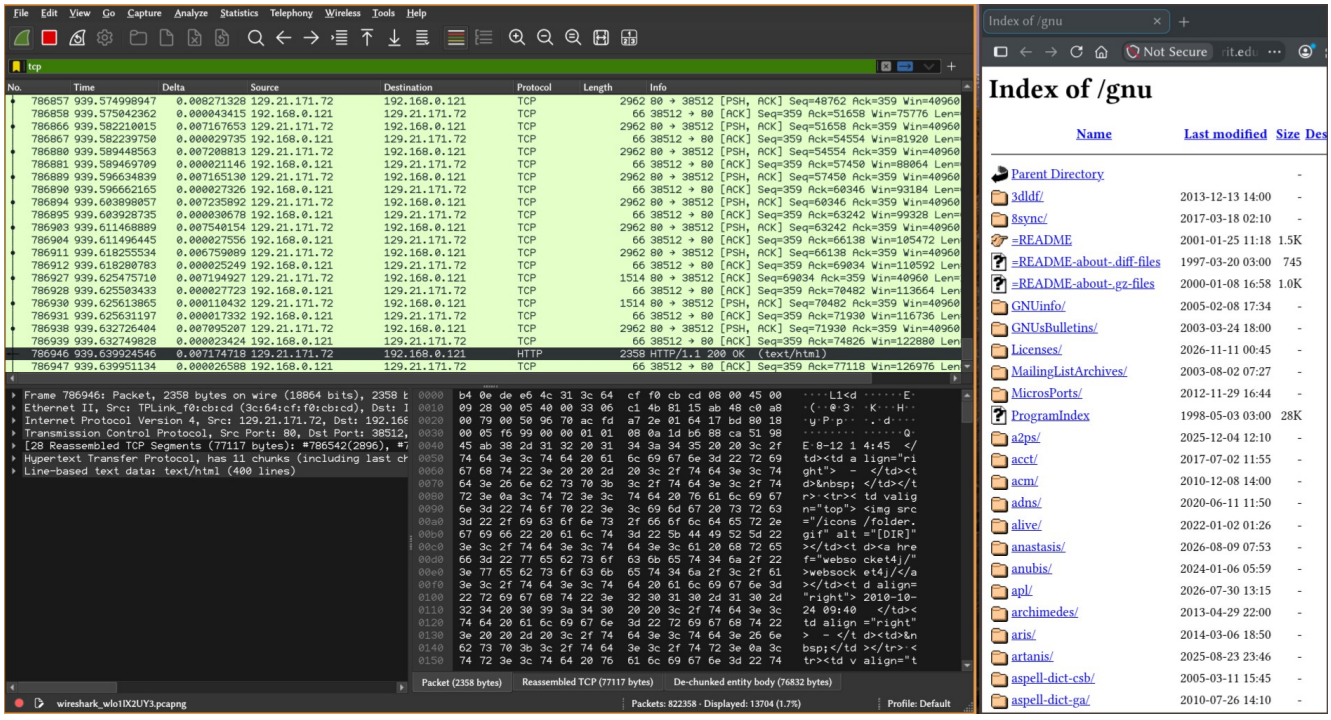


Рисунок 2 - http трафік

Як видно, http трафік не зашифровано, і html документ, що він передав можна легко передивитися у змісті паекту

3 Налаштування брандмауера:

- Переглянути поточні правила брандмауера
- Створити правило для блокування порту з варіанта
- Створити правило для дозволу порту з варіанта
- Перевірити роботу правил
- Видалити тестові правила

```
$ doas iptables -L -n -v
Chain INPUT (policy DROP 1162 packets, 40444 bytes)
 pkts bytes target     prot opt in     out     source               destination
  11M   20G ufw-before-logging-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
  11M   20G ufw-before-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
10312 3553K ufw-after-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
 1162 40444 ufw-after-logging-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
 1162 40444 ufw-reject-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0
 1162 40444 ufw-track-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ufw-before-logging-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ufw-before-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ufw-after-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ufw-after-logging-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ufw-reject-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ufw-track-forward  all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain OUTPUT (policy ACCEPT 995 packets, 35226 bytes)
 pkts bytes target     prot opt in     out     source               destination
1851K  575M ufw-before-logging-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
1851K  575M ufw-before-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
37291   17M ufw-after-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
37291   17M ufw-after-logging-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
37291   17M ufw-reject-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0
37291   17M ufw-track-output  all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-after-forward (1 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-after-input (1 references)
 pkts bytes target     prot opt in     out     source               destination
  261 23598 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:137
  184 42136 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:138
    0     0 ufw-skip-to-policy-input  tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:139
    0     0 ufw-skip-to-policy-input  tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:445
    0     0 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:67
    0     0 ufw-skip-to-policy-input  udp  --  *      *       0.0.0.0/0            0.0.0.0/0           udp dpt:68
 8705 3447K ufw-skip-to-policy-input  all  --  *      *       0.0.0.0/0            0.0.0.0/0           ADDRTYPE match dst-ty
pe BROADCAST

Chain ufw-after-logging-forward (1 references)
```

Рисунок 3 - правила iptables

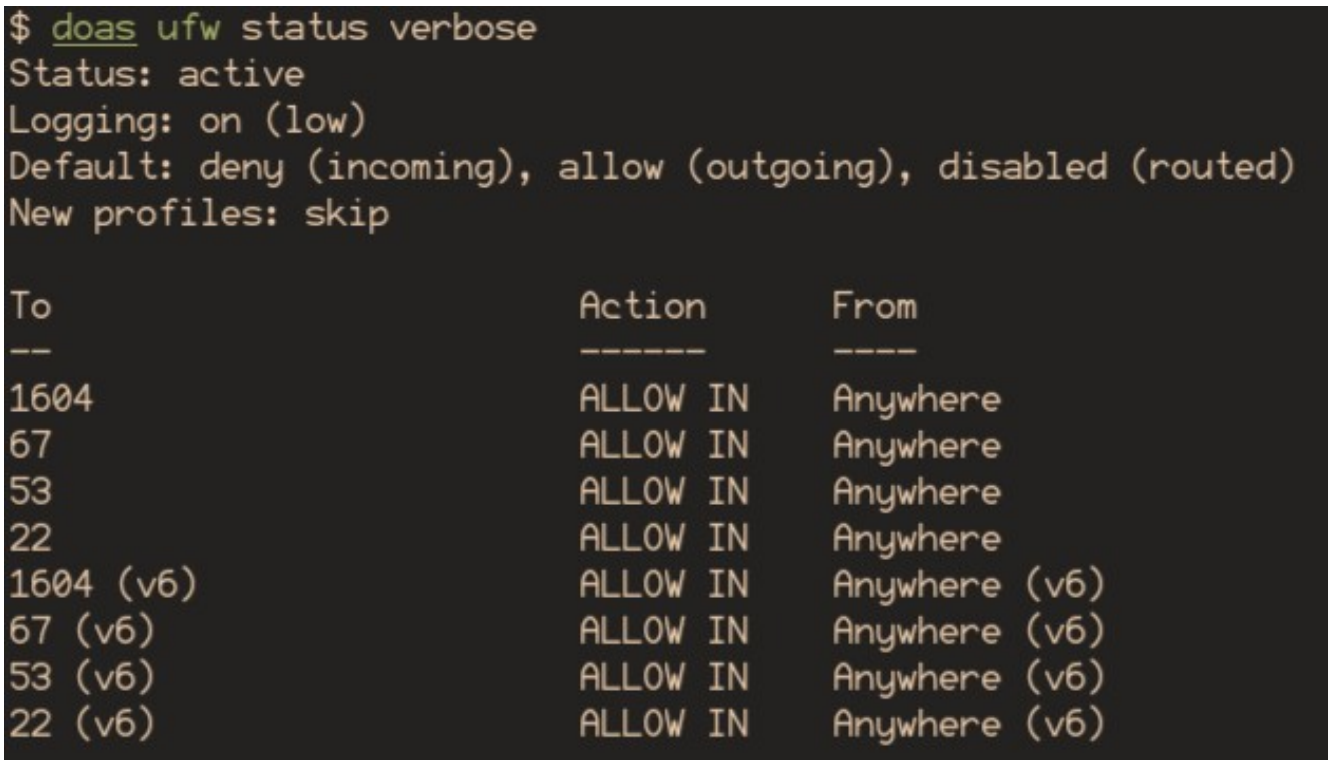


Рисунок 4 - правила ufw

```
[~] <yurii>
$ doas ufw allow 3389/tcp
Rule added
Rule added (v6)

[~] <yurii>
$ doas ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
1604                       ALLOW IN    Anywhere
67                         ALLOW IN    Anywhere
53                         ALLOW IN    Anywhere
22                         ALLOW IN    Anywhere
135/tcp                    REJECT IN   Anywhere
3389/tcp                   ALLOW IN    Anywhere
1604 (v6)                  ALLOW IN    Anywhere (v6)
67 (v6)                    ALLOW IN    Anywhere (v6)
53 (v6)                    ALLOW IN    Anywhere (v6)
22 (v6)                    ALLOW IN    Anywhere (v6)
135/tcp (v6)               REJECT IN   Anywhere (v6)
3389/tcp (v6)              ALLOW IN    Anywhere (v6)
```

Рисунок 5 - налаштування правил

```
$ doas nc -vl 0.0.0.0 135
Listening on 0.0.0.0 135
```

Рисунок 6 - сервіс на порті 135

```
$ nc -v 192.168.0.121 135
nc: connect to 192.168.0.121 port 135 (tcp) failed: Connection refused
```

Рисунок 7 - спроба під’єднання до порту 135 на віддаленій машині

```
$ nc -vl 0.0.0.0 3389
Listening on 0.0.0.0 3389
Connection received on yurinpc.local 45140
hello there
```

Рисунок 8 - сервіс на порту 3389

```
$ nc -v 192.168.0.121 3389
Connection to 192.168.0.121 3389 port [tcp/ms-wbt-server] succeeded!
hello there
```

Рисунок 9 - під’єднання до сервісу на порту 3389

4 Сканування портів:

- Визначити відкриті порти на localhost
- Ідентифікувати процеси, що прослуховують порти
- Записати результати

```
$ ss -tulnp | grep LISTEN
tcp   LISTEN 0      0          127.0.0.1:6463       0.0.0.0:*    users:(("Discord",pid=323556,fd=82))
tcp   LISTEN 0      0            0.0.0.0:5355       0.0.0.0:*
tcp   LISTEN 0      0         127.0.0.53:53         0.0.0.0:*
tcp   LISTEN 0      0          127.0.0.1:27017      0.0.0.0:*    users:(("mongod",pid=300059,fd=8))
tcp   LISTEN 0      0          127.0.0.1:11434      0.0.0.0:*    users:(("ollama",pid=301342,fd=4))
tcp   LISTEN 0      0         127.0.0.54:53         0.0.0.0:*
tcp   LISTEN 0      0              [::1]:5173             *:*    users:(("node-MainThread",pid=326311,fd=25))
tcp   LISTEN 0      0                 *:5355             *:*
```

Рисунок 10 - під’єднання до сервісу на порту 3389

5 Захист веб-застосунку:

– Запустити демонстраційний код SQL-ін’єкції
– Продемонструвати успішну атаку на вразливий код
– Продемонструвати блокування атаки безпечним кодом
– Повторити для XSS

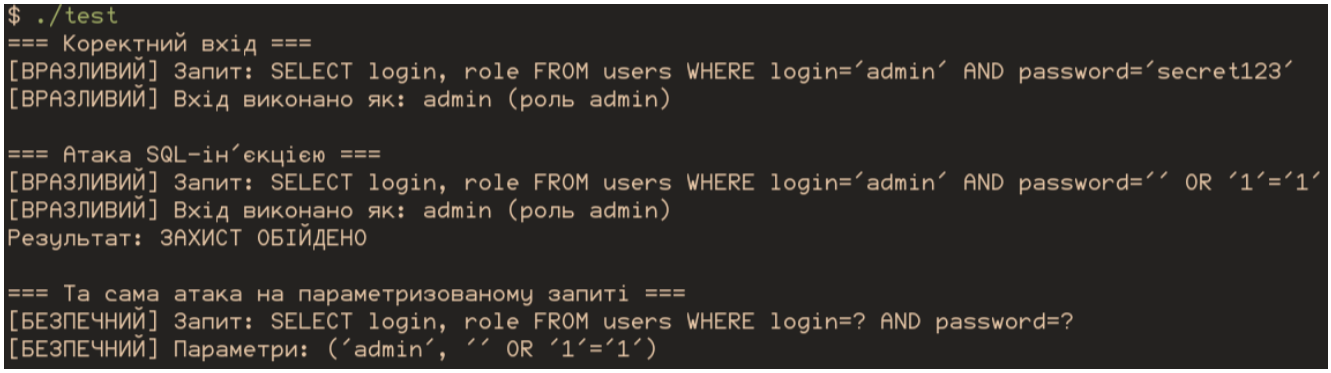


Рисунок 11 - тестування sql ін’єкцій

6 Виконати індивідуальне завдання згідно з варіантом.

7 Оформити звіт та зробити висновок.

Відповіді на контрольні питання

Високий рівень (творчий)

1. Побудуйте ешелоновану схему захисту веб-застосунку: які засоби на якому рівні та проти яких загроз працюють.

1. ОС: використовувати надійну операційну систему (OpenBSD чи hardened linux для критичних застосунків, звичайний linux чи bsd як стандарт)
2. Оточення: firewall, fail2ban, ssh, nginx та інші речі, що не прив’язуються напряму до програми, але при правильному налаштуванні надає кращий захист
3. Рівень програми: використовувати надійні технології як основу (точно не PHP та JavaScript), якщо потрібна більша швидкість, то надавати

перевагу memory-safe мовам (go, rust, …). Використовувати перевірені бібліотеки та фреймворки, мінімізувати кількість залежностей, розглянути можливість vendor-locking.

4. Архітектура: передбачити авторизацію, надійне зберігання паролів, використовувати базу даних таким чином, щоб sql ін’єкції не були проблемою.

2. Система виявлення вторгнень дає багато хибних спрацьовувань. Назвіть причини та порядок налаштування, щоб їх зменшити без втрати чутливості.

Тому що такі системи працюють по певним евристичним алгоритмам. Наприклад блокування ip адресс, що надсилають занадто багато запитів. Але такий трафік може бути результатом багатьох інших факторів окрім спроби атаки.

Ручний аналіз трафіку на вже запущеному сервесі та відповідний підгін конфігурації може зменшити кількість хибних спрацювань, але усеодно не усунути проблему повністю.

3. Оцініть, чому повне блокування вхідного трафіку не є універсальним розв’язанням, і сформулюйте критерій відкриття порту.

Як мінімум тому, що інтернет-сервіс, що блокує весь вхідний трафік не зможе нічого робити. Щось треба відкривати для надання функціоналу. Відкривати порти треба по принциму мінімально необхідного об’єму: заблокувати все, а потім додавати виключення у firewall для потрібних ресурсів.

4. Сформулюйте, чому більшість успішних атак використовує не технічні вразливості, а помилки конфігурації та людський фактор. Проілюструйте прикладами з роботи.

Тому що станом на зараз software частина є достатньо надійною, щоб атака на пряму вразливість була надто складною чи взагалі неможливою. Люди ж, які налаштовують та використовують сервіси не завжди можуть захистити та зробити усе правильно, тому такі атаки значно простіші.

Більшість великих відомих атак більше про соціальний інженеринг, ніж про комп’ютерний.

Із появою та широкою інтеграцією AI зараз він став найвразливішою ланкою, але людський фактор все ще досить критична частина.

Висновок: Я зрозумів, чому наявність https є критичною для безпеки, як можна налаштовувати правила firewall-у, як ідентифікувати процеси, що займають сокети у системі та принцип SQL-ін’єкцій.